A Mini Project Report

on

Multilingual Document Summarizer

Submitted to

AISSMS College of Engineering, Pune

In partial fulfillment of the requirements for the Third Year of the Degree of

BE/BTech

in

Artificial Intelligence and Machine Learning

Submitted by

1. Hardik Pathak – _____
2. _____
3. _____

Under the guidance of

Prof. _____

Department of Artificial Intelligence and Machine Learning

AISSMS College of Engineering

Kennedy Road, Pune – 411001, Maharashtra, India

Academic Year: 2026–2027

CERTIFICATE

This is to certify that the project report entitled “Multilingual Document Summarizer” is a record of bonafide work carried out by the students under the supervision and guidance of Prof. _____, in partial fulfillment of the requirements for the Third Year (Artificial Intelligence and Machine Learning) programme of AISSMS College of Engineering, Pune, during the academic year 2026–2027.

Date:

Place: Pune

Submitted by:

1. Hardik Pathak – _____
2. _____
3. _____

_____ _____

Prof. _____ Dr. _____

Project Guide Head of Department

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to our project guide for providing valuable guidance, encouragement, and support throughout the development of our mini project, “Multilingual Document Summarizer”. Their valuable suggestions, constructive feedback, and continuous motivation helped us understand the project requirements and complete the work successfully.

We are thankful to our Head of Department and all faculty members for giving us the opportunity to work on this project as part of our Natural Language Processing (NLP) curriculum. Their guidance and support encouraged us to explore the practical applications of Natural Language Processing.

We would also like to extend our sincere thanks to our college for providing the necessary facilities, resources, and a positive learning environment throughout the project development process.

Through this project, we gained practical experience in text preprocessing, abstractive summarization using Transformer-based models (BART and IndicBART), Optical Character Recognition, model fine-tuning on Kaggle GPUs, ROUGE-based evaluation, and web application development using Python, FastAPI, and Streamlit.

We would also like to acknowledge the open-source datasets and models (CNN/DailyMail, XL-Sum, Hugging Face Transformers, EasyOCR) that helped us understand the tools used in this project.

Finally, we are thankful to everyone who directly or indirectly supported us during the completion of this mini project. We hope that the system can contribute towards easier access to long documents for readers of English and Hindi.

ABSTRACT

Multilingual Document Summarizer is a web-based application that produces concise, abstractive summaries of long documents and images in English and Hindi. Large amounts of useful information are locked inside PDF files, Word documents, plain-text files, and scanned images, and readers often need to spend considerable time locating the key points. Language is an additional barrier, because most existing summarization tools support English only.

The system accepts .txt, .pdf, and .docx files, images (.png, .jpg), and pasted text. Document text is extracted using PyMuPDF and python-docx, and image text is extracted using OCR (EasyOCR locally, or Gemini vision in the cloud). The extracted text is passed to a language-specific sequence-to-sequence model that generates an abstractive summary. English documents are summarized using BART fine-tuned on CNN/DailyMail, and Hindi documents using IndicBART fine-tuned on XL-Sum.

The application is developed using Python and the Hugging Face Transformers library. The user interface is provided as a FastAPI-backed web application with an HTML/CSS/JavaScript frontend and as a Streamlit demo. The models were fine-tuned on an NVIDIA T4 GPU and evaluated using ROUGE. An optional Gemini API backend allows the app to run without downloading local models.

The main objective of the project is to make long documents easier to understand and to make summarization available to readers of Hindi as well as English.

Keywords: Natural Language Processing (NLP), Abstractive Summarization, Tokenization, BART, IndicBART, XL-Sum, Optical Character Recognition (OCR), Multilingual, Fine-tuning, ROUGE, FastAPI, Streamlit, Gemini API.

CONTENT

Chapter / Contents

Acknowledgement i

Abstract ii

Table of Contents iii

List of Figures iv

List of Tables v

Chapter 1: Introduction 1

1.1 Introduction to the Project 1

1.2 Background and Motivation 1

1.3 Problem Statement 2

1.4 Objectives 2

1.5 Scope of the Project 2

1.6 Applications of the Project 3

Chapter 2: Literature Review 4

2.1 Review of Existing Research Papers 4

2.2 Existing System 5

2.3 Limitations of Existing System 5

2.4 Research Gap 5

Chapter 3: Problem Statement and Requirements 6

3.1 Problem Definition 6

3.2 Proposed Solution 6

3.3 Functional Requirements 6

3.4 Non-Functional Requirements 7

3.5 Hardware and Software Requirements 7

Chapter 4: Proposed Methodology 8

4.1 Overview of the Proposed System 8

4.2 System Architecture 8

4.3 Workflow of the System 9

4.4 NLP Techniques and Algorithms 9

4.5 Dataset Collection and Preprocessing 11

Chapter 5: System Design and Implementation 12

5.1 System Design 12

5.2 Flowchart / UML / DFD Diagrams 12

5.3 Tools, Technologies and Libraries 13

5.4 Implementation Details 13

5.5 User Interface and Screenshots 14

Chapter 6: Results and Discussion 15

6.1 Experimental Setup 15

6.2 Test Cases and Sample Outputs 15

6.3 Performance Evaluation Metrics 16

6.4 Results and Observations 16

6.5 Limitations 17

Chapter 7: Conclusion and Future Scope 18

7.1 Conclusion 18

7.2 Future Scope 18

References 19

Appendix A: Important Code Snippets 20

Appendix B: Additional Screenshots / Outputs 21

LIST OF FIGURES

Figure Title	Page No.
Fig. 1.1: Overview of the Summarization Pipeline	3
Fig. 2.1: Existing Summarization Workflow	6
Fig. 4.1: Proposed System Architecture	8
Fig. 4.2: Workflow of the Proposed System	9
Fig. 5.1: General System Flowchart	12
Fig. 5.2: Data Flow Diagram (Level 0)	12
Fig. 5.3: Web Application User Interface	14
Fig. A.1: Summarization function	20
Fig. A.2: OCR routing	20
Fig. A.3: Image summarization endpoint	21
Fig. A.4: Gemini vision client	21

LIST OF TABLES

Table Title	Page No.
Table 2.1: Summary of Literature Review	5
Table 3.1: Functional Requirements	6
Table 3.2: Non-Functional Requirements	7
Table 3.3: Hardware Requirements	7
Table 3.4: Software Requirements	8
Table 4.1: Dataset Description	11
Table 5.1: Tools, Technologies and Libraries	13
Table 6.1: Functional Test Cases	15
Table 6.2: ROUGE Score Results	16

CHAPTER 1
INTRODUCTION

1.1 Introduction to the Project:

The amount of digital text available today is enormous. Long PDF documents, Word files, reports, and articles are used for education, news, research, business, and public administration. Reading every document completely is time-consuming, and important points can be difficult to locate quickly. A summarization system helps a reader understand the key ideas of a document in a short time.

Multilingual Document Summarizer is an NLP-based application that generates abstractive summaries of documents and images in English and Hindi. Documents can be uploaded as .txt, .pdf, or .docx files, images can be uploaded as .png or .jpg, and users can also paste text directly.

The extracted text is sent to a Transformer-based sequence-to-sequence model. English text is summarized using BART fine-tuned on CNN/DailyMail, and Hindi text using IndicBART fine-tuned on XL-Sum. The system is exposed through a web application built with FastAPI and an HTML/CSS/JavaScript frontend, and through a Streamlit demo.

1.2 Background and Motivation

Digital technology has changed the way people read and share information. Documents are increasingly available online, and mobile cameras make it easy to capture text as images. Readers often need a quick understanding of a long document rather than a complete reading, which motivates automatic summarization.

Language is another important consideration. Many readers in India are more comfortable reading Hindi than English, and most summarization tools support only English. Providing summaries in Hindi makes useful information accessible to a wider audience.

Finally, a large amount of content is available only as images or scanned documents. Combining OCR with summarization allows such content to be handled by the same system.

1.3 Problem Statement

To develop a multilingual NLP-based system that accepts documents and images, extracts their text (using OCR for images), and generates abstractive summaries in English and Hindi through a simple web interface, so that readers can understand long content quickly without being limited by language or input format.

1.4 Objectives

The major objectives of the project are:

1. To develop a system that summarizes long documents and images in English and Hindi.
2. To implement OCR for extracting text from images.
3. To fine-tune Transformer-based models (BART and IndicBART) for abstractive summarization.
4. To support .txt, .pdf, .docx, and image inputs, as well as pasted text.
5. To provide a clean web interface (FastAPI + HTML/CSS/JS) and a Streamlit demo.
6. To evaluate the fine-tuned models using the ROUGE metric.

7. To provide an optional Gemini API backend that runs without local models.

1.5 Scope of the Project

The project focuses on building a working bilingual summarization tool. Its scope includes:

- Uploading .txt, .pdf, and .docx documents and extracting their text.
- Uploading .png/.jpg images and extracting text using OCR.
- Pasting text directly for summarization.
- Generating abstractive summaries in English and Hindi.
- Fine-tuning BART (English) and IndicBART (Hindi).
- Evaluating quality with ROUGE-1, ROUGE-2, ROUGE-L, and ROUGE-Lsum.
- A FastAPI web application, a Streamlit demo, and an optional Gemini backend.

Summaries of very long documents are limited by the model's maximum input length.

1.6 Applications of the Project

1. Education: students can summarize long study material and scanned notes.
2. News and Media: quick summaries of long articles in English or Hindi.
3. Government and Administration: summarizing lengthy notices and reports.
4. Accessibility: Hindi readers can access content in their own language.
5. Research and Business: fast scanning of reports and documents.
6. Digital Libraries: converting scanned pages and long PDFs into short summaries.

CHAPTER 2
LITERATURE REVIEW

2.1 Review of Existing Research Papers

Natural Language Processing, abstractive summarization, Transformer models, and OCR are the main research areas related to this project.

2.1.1 Attention Is All You Need

Vaswani et al. (2017) introduced the Transformer architecture, which uses self-attention to model relationships between elements of a sequence. Transformers are the foundation of BART and IndicBART.

2.1.2 BART: Denoising Sequence-to-Sequence Pre-training

Lewis et al. (2019) introduced BART, a denoising autoencoder that combines bidirectional and autoregressive Transformers. BART is effective for abstractive summarization and bart-large-cnn is a standard baseline on CNN/DailyMail.

2.1.3 IndicBART for Indian Languages

Dabre et al. (2022) introduced IndicBART, a pre-trained sequence-to-sequence model for Indian languages. It uses a compact mBART-style architecture and a language-token format that allows fine-tuning for Hindi summarization.

2.1.4 XL-Sum: Large-Scale Multilingual Abstractive Summarization

Hasan et al. (2021) introduced XL-Sum, a multilingual dataset of news article and summary pairs covering 44 languages, including Hindi. It is used to fine-tune the Indic model.

2.1.5 OCR for Text Extraction from Images

Optical Character Recognition converts text in images into machine-readable text. EasyOCR is a deep-learning OCR library supporting English and Hindi, and modern vision models such as Gemini can also perform OCR.

Table 2.1: Summary of Literature Review

Research / resource	Main contribution	Relevance
Vaswani et al. (2017)	Transformer architecture	Foundation of BART and IndicBART
Lewis et al. (2019)	BART denoising model	English abstractive summarizer
Dabre et al. (2022)	IndicBART for Indian languages	Hindi summarization
Hasan et al. (2021)	XL-Sum multilingual dataset	Fine-tuning the Hindi model
EasyOCR / Gemini	OCR for images	Text acquisition from images

2.2 Existing System

Existing summarization tools mostly target English text. They include cloud-based summarization APIs, browser extensions, and standalone applications. Some tools also support OCR, but text extraction and summarization are usually provided by separate systems.

Indic summarization is less commonly available. Integrated document-and-image summarization for Hindi is not readily available to general users.

2.3 Limitations of Existing System

1. Most summarization tools support English only.
2. Regional languages such as Hindi are poorly supported.
3. Text extraction from images and summarization are usually separate tools.
4. Uploading long documents in multiple formats is not always supported.
5. Many tools require programming knowledge or paid API access.

2.4 Research Gap

There is a need for a unified system that accepts both documents and images, supports English and Hindi, and provides summaries through a simple web interface. This project fills that gap by combining fine-tuned BART and IndicBART models, OCR, and a user-friendly web front-end in a single application.

CHAPTER 3
PROBLEM STATEMENT AND REQUIREMENTS

3.1 Problem Definition

The problem addressed by the project is the difficulty readers face in quickly understanding long documents and images, especially when the content is in a regional language. A user requires a simple method to upload content, choose a language, and receive a short summary without installing machine-learning libraries.

3.2 Proposed Solution

The user uploads a document or an image, or pastes text, and selects a language. The backend extracts the text, passes it to the summarization model, and returns the summary.

The system uses Python and Hugging Face Transformers, with PyMuPDF and python-docx for text extraction and EasyOCR (or Gemini vision) for OCR. A Streamlit version and an optional Gemini API backend are also provided.

3.3 Functional Requirements

Table 3.1: Functional Requirements

ID	Requirement	Description
FR01	Upload document	Accept .txt, .pdf, and .docx files
FR02	Upload image	Accept .png and .jpg files
FR03	Text extraction	Extract text from the document or image (OCR)
FR04	Language selection	Allow the user to choose English or Hindi
FR05	Summarize text	Generate an abstractive summary
FR06	Display result	Show the summary and the extracted text
FR07	Download summary	Allow the summary to be downloaded as a text file
FR08	Model selection	Automatically use the fine-tuned model if available
FR09	Error handling	Display friendly errors for unsupported or empty input

3.4 Non-Functional Requirements

Table 3.2: Non-Functional Requirements

Requirement	Description
Usability	The interface should be simple and easy to navigate
Performance	Summaries should be generated in a reasonable time
Reliability	Valid inputs should produce a summary without crashing
Maintainability	Code should be organized into clear modules
Scalability	Additional languages and models should be easy to add
Compatibility	The web interface should work in modern browsers
Data privacy	Uploaded content should be processed securely

3.5 Hardware and Software Requirements

Table 3.3: Hardware Requirements

Component	Requirement
Processor	Intel Core i5 or equivalent (Apple M-series supported)
RAM	8 GB minimum; 16 GB recommended for model loading
Storage	Approximately 5 GB for models and dependencies
Display	Standard laptop or desktop display
Network	Internet required for the first model download

Table 3.4: Software Requirements

Software	Purpose
Python 3.10+	Programming language
PyTorch	Deep-learning backend
Hugging Face Transformers	BART and IndicBART models
Hugging Face Datasets	CNN/DailyMail and XL-Sum
EasyOCR	Optical Character Recognition
PyMuPDF / python-docx	PDF and DOCX text extraction
FastAPI + Uvicorn	Web backend
Streamlit	Demo interface
HTML / CSS / JavaScript	Web frontend
Evaluate / rouge_score	ROUGE evaluation
Google Gemini API (optional)	Hosted text + vision backend

CHAPTER 4
PROPOSED METHODOLOGY

4.1 Overview of the Proposed System

The frontend sends the uploaded file or pasted text and the selected language to the backend. The backend performs text extraction, selects the appropriate model, generates a summary, and returns it to the frontend.

1. Frontend: upload interface, language selection, and result display.
2. Text extraction: PDF, DOCX, TXT parsing, and OCR for images.
3. Summarization: language-specific BART or IndicBART inference (or the Gemini API).
4. Training pipeline: fine-tuning scripts and the Kaggle notebooks.

4.2 System Architecture

1. Presentation layer: HTML/CSS/JavaScript (FastAPI) and Streamlit.
2. Extraction layer: PyMuPDF, python-docx, and EasyOCR / Gemini vision.
3. Model layer: BART (English) and IndicBART (Hindi), loaded from models/ if present.
4. Optional API layer: Google Gemini, used when a Gemini key is configured.
5. Training layer: Kaggle notebooks and scripts/train.py.

For Hindi, the input is formatted as "<text> </s> <2hi>" and the decoder is forced to start with the language token, as required by IndicBART.

4.3 Workflow of the System

1. The user opens the web application.
2. The user uploads a document/image or pastes text and selects a language.
3. The backend extracts text using the appropriate extractor or OCR.
4. The text is passed to the language-specific model (or the Gemini API).
5. The model generates an abstractive summary using beam search.
6. The summary and extracted text are returned to the frontend.
7. The user can copy or download the summary.

4.4 NLP Techniques and Algorithms

Natural Language Processing is used at every stage of this project. The following sub-sections describe each NLP technique and where it is applied.

4.4.1 Text Preprocessing and Tokenization

Raw text is cleaned and split into sub-word tokens using the BART and IndicBART tokenizers. Tokenization converts human-readable text into the integer token ids that the model processes, and truncation ensures the input fits the model's maximum length.

4.4.2 Sequence-to-Sequence Abstractive Summarization

The core NLP task is abstractive summarization. An encoder reads the input document and a decoder generates a new, shorter text that captures the main ideas. BART is used for English and IndicBART for Hindi, after fine-tuning on summarization datasets.

4.4.3 Beam Search Decoding

Summaries are generated token-by-token using beam search (num_beams = 4-5), combined with no-repeat n-gram constraints and length penalties. This produces fluent summaries while avoiding repetition and controlling length.

4.4.4 Indic Language Handling

IndicBART uses language tokens. The input is appended with "</s>" and a language token such as <2hi>, the target is prefixed with the same token, and the decoder start token is forced to it. This ensures that the generated summary is in Devanagari and in the correct language.

4.4.5 OCR as an NLP Text-Acquisition Step

Images are first converted into text using OCR. EasyOCR is used in local mode, and Gemini vision is used in the cloud backend. The recognized text then enters the same NLP summarization pipeline as a normal document.

4.4.6 Evaluation with ROUGE

Model quality is measured with ROUGE, which compares the generated summary with a reference summary using unigram overlap (ROUGE-1), bigram overlap (ROUGE-2), and longest common subsequence (ROUGE-L / ROUGE-Lsum).

4.4.7 Summary: Role of NLP in the Project

- Tokenization converts documents into model-readable token sequences.
- Sequence-to-sequence models perform abstractive summarization in English and Hindi.
- Beam search and n-gram constraints control the quality and length of generated text.
- IndicBART language tokens enable correct Devanagari output for Hindi.
- OCR supplies text from images so that it can be processed by the NLP pipeline.
- ROUGE provides a quantitative measure of summary quality.
- Fine-tuning adapts large pre-trained language models to the summarization task.

4.5 Dataset Collection and Preprocessing

The project uses two public datasets:

- CNN/DailyMail (English): news articles paired with human-written highlights.
- XL-Sum (Hindi): news articles paired with summaries.

XL-Sum is distributed as a script-based dataset, which newer versions of the Datasets library no longer support, so the project loads the auto-converted Parquet shards directly. Preprocessing includes tokenization, truncation, and, for IndicBART, language-token formatting.

Table 4.1: Dataset Description

Dataset	Language	Task	Usage
CNN/DailyMail	English	News summarization	Fine-tuning BART
XL-Sum (hindi)	Hindi	News summarization	Fine-tuning IndicBART

CHAPTER 5
SYSTEM DESIGN AND IMPLEMENTATION

5.1 System Design

The system is divided into independent modules to simplify development and maintenance.

- Input and upload interface.
- Document text extraction (PyMuPDF, python-docx).
- Image text extraction (EasyOCR / Gemini vision).
- Language-aware summarization (BART / IndicBART).
- Optional Gemini API backend.
- Web API layer (FastAPI) and Streamlit demo.

The modules share a common src/ package, so the web app, Streamlit demo, and training script use the same inference code.

5.2 Flowchart / UML / DFD Diagrams

The following diagrams describe the proposed system. They can be inserted using draw.io, Microsoft Word Shapes, or another diagram tool.

5.2.1 General System Flowchart

Upload → text extraction / OCR → language selection → model inference → summary → display/download.

Figure 5.1: General workflow of the bilingual summarization system.

5.2.2 Data Flow Diagram (Level 0)

The user provides a document or image and a language; the system returns a summary. The model, OCR, and text-extraction components are internal processes.

Figure 5.2: Level 0 Data Flow Diagram.

5.3 Tools, Technologies and Libraries

Table 5.1: Tools, Technologies and Libraries

Technology	Application
Python	Core programming language
PyTorch	Deep-learning backend
Hugging Face Transformers	BART and IndicBART models
Hugging Face Datasets	CNN/DailyMail and XL-Sum loading
EasyOCR	Image text extraction (local)
PyMuPDF	PDF text extraction
python-docx	DOCX text extraction
FastAPI + Uvicorn	Web backend
Streamlit	Demo interface
HTML / CSS / JavaScript	Web frontend
Evaluate / rouge_score	ROUGE evaluation
Google Gemini API (optional)	Hosted text + vision backend

5.4 Implementation Details

5.4.1 Inference Core (src/)

config.py defines the model registry, provider selection, and cache handling. text_extraction.py extracts text from .txt, .pdf, and .docx. ocr.py extracts text from images using EasyOCR or Gemini vision. summarizer.py loads the language-specific model and generates the summary, and llm_api.py implements the Gemini REST client.

5.4.2 Web Application

The FastAPI backend (web/server.py) exposes endpoints for text, document, and image summarization and serves the static frontend (web/static/), which provides drag-and-drop upload, language and length controls, and copy/download actions.

5.4.3 Streamlit Demo

app.py provides the same functionality through a sidebar-driven Streamlit interface with Document, Image, and Text modes.

5.4.4 Training Pipeline

The models are fine-tuned with scripts/train.py and the notebooks in notebooks/. Training uses the Hugging Face Seq2SeqTrainer, and evaluation uses ROUGE-1, ROUGE-2, ROUGE-L, and ROUGE-Lsum.

5.5 User Interface and Screenshots

The web application provides a clean interface for uploading documents and images and for selecting the summary language. A screenshot of the running application is shown below.

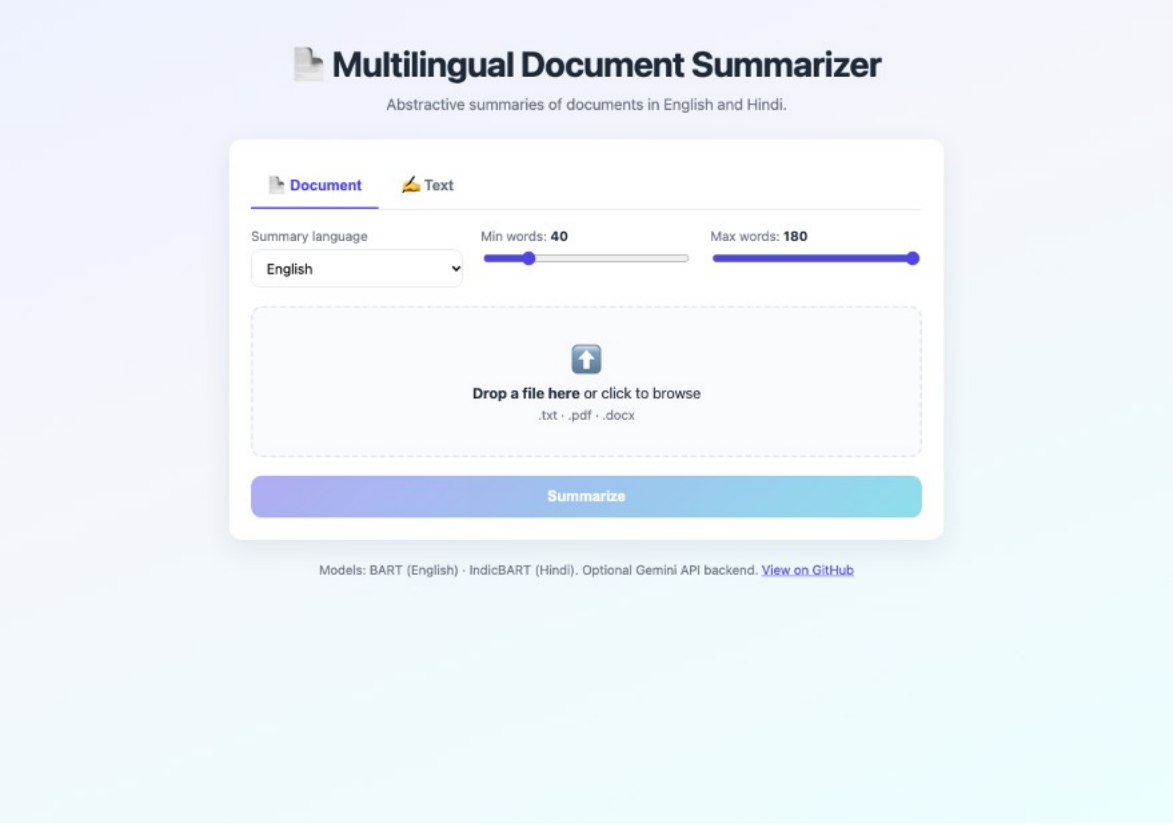


Figure 5.3: Web Application User Interface

CHAPTER 6
RESULTS AND DISCUSSION

6.1 Experimental Setup

The models were fine-tuned on a Kaggle NVIDIA T4 GPU. BART-base was fine-tuned on CNN/DailyMail and IndicBART on the Hindi subset of XL-Sum. For each language, 2,000 training examples and 200 validation examples were used for two epochs, with a batch size of 4 and a learning rate of 3e-5. Summaries were generated with beam search and evaluated with ROUGE.

6.2 Test Cases and Sample Outputs

Table 6.1: Functional Test Cases

Test ID	Input / Action	Expected output
TC01	Open the web application	Upload interface is displayed
TC02	Upload a valid .txt file	Text is extracted and shown
TC03	Upload a valid .pdf file	Text is extracted and shown
TC04	Upload a valid .docx file	Text is extracted and shown
TC05	Upload a valid image	OCR text is extracted and shown
TC06	Paste text and summarize	Summary is generated
TC07	Select English and summarize	English summary is generated
TC08	Select Hindi and summarize	Hindi summary is generated
TC09	Upload an empty/unsupported file	A friendly error is shown
TC10	Download the summary	A .txt file is downloaded

6.3 Performance Evaluation Metrics

- ROUGE-1: unigram overlap between the generated and reference summaries.
- ROUGE-2: bigram overlap.
- ROUGE-L: longest common subsequence.
- ROUGE-Lsum: sentence-level longest common subsequence.

6.4 Results and Observations

The fine-tuned models produced the following validation results:

Table 6.2: ROUGE Score Results

Language	Model	ROUGE-1	ROUGE-2	ROUGE-L	ROUGE-Lsum
English	BART-base (CNN/DailyMail)	40.65	19.10	28.30	37.59
Hindi	IndicBART (XL-Sum)	60.42	25.90	60.36	60.26

The English model produced concise and relevant summaries. The Hindi model produced coherent summaries in the Devanagari script. Because only 2,000 examples were used, the Hindi model tends to be verbose and occasionally repeats phrases; this can be improved with more training data.

The application was verified end-to-end: documents and images were uploaded, text was extracted (including OCR), and summaries were generated in both languages. Fine-tuned weights placed in models/ are detected automatically and used instead of the public checkpoints.

6.5 Limitations

1. The Hindi model was fine-tuned on a small sample and can be verbose.
2. Very long documents exceed the model's maximum input length and are truncated.
3. OCR accuracy depends on image quality and font rendering.
4. The local models require a ~2 GB download on first use.
5. The system runs on CPU by default; GPU improves speed.

CHAPTER 7
CONCLUSION AND FUTURE SCOPE

7.1 Conclusion

Multilingual Document Summarizer is an NLP-based application that makes long documents and images easier to understand in English and Hindi. The project integrates text extraction (PDF, DOCX, TXT), OCR, abstractive summarization using BART and IndicBART, a FastAPI web application, and a Streamlit demo.

The models were fine-tuned on Kaggle GPUs and evaluated using ROUGE. The English model achieved a ROUGE-1 of 40.65 and the Hindi model 60.42, showing that the approach is practical. The project demonstrates how NLP, OCR, and web technologies can be combined to build a useful bilingual tool.

7.2 Future Scope

1. Fine-tune on the full datasets and more epochs for higher-quality summaries.
2. Add chunking and hierarchical summarization for very long documents.
3. Support additional Indian languages and scripts.
4. Improve the Hindi model to reduce repetition and verbosity.
5. Add voice input and text-to-speech for accessibility.
6. Deploy the application on a public server with authentication.

REFERENCES

1. Lewis, M., et al. (2019). “BART: Denoising Sequence-to-Sequence Pre-training.” arXiv:1910.13461.
2. Dabre, R., et al. (2022). “IndicBART: A Pre-trained Model for Natural Language Generation of Indic Languages.” arXiv:2109.02903.
3. Hasan, T., et al. (2021). “XL-Sum: Large-Scale Multilingual Abstractive Summarization for 44 Languages.” Findings of ACL-IJCNLP 2021.
4. Vaswani, A., et al. (2017). “Attention Is All You Need.” Advances in Neural Information Processing Systems.
5. Hermann, K. M., et al. (2015). “Teaching Machines to Read and Comprehend.” NeurIPS. (CNN/DailyMail)
6. EasyOCR. https://github.com/JaidedAI/EasyOCR
7. Hugging Face Transformers. https://huggingface.co/docs/transformers/
8. PyMuPDF Documentation. https://pymupdf.readthedocs.io/
9. FastAPI Documentation. https://fastapi.tiangolo.com/
10. Streamlit Documentation. https://docs.streamlit.io/
11. Google Gemini API. https://ai.google.dev/

APPENDIX A: IMPORTANT CODE SNIPPETS

This appendix presents the core code modules of the project as editor screenshots, followed by a short description of each.

A.1 Summarization Function

Source file: src/summarizer.py

src/summarizer.py

```python
def summarize(text, language, max_length=None, min_length=None):
    from .config import SUMMARY_PROVIDER
    if SUMMARY_PROVIDER == "gemini":
        from .llm_api import summarize_with_gemini
        return summarize_with_gemini(text, language, max_length=max_length,
                                     min_length=min_length)
    import torch
    cfg = SUMMARIZATION_MODELS[language]
    tokenizer, model = _load_pipeline(language)
    gen_kwargs = dict(cfg["gen_kwargs"])
    lang_code = cfg.get("lang_code")
    if lang_code:
        inputs = _indic_inputs(tokenizer, model, text, lang_code,
                               cfg["max_input_tokens"])
    else:
        inputs = tokenizer(text, max_length=cfg["max_input_tokens"],
                           truncation=True, return_tensors="pt").to(model.device)
    with torch.no_grad():
        output_ids = model.generate(**inputs, **gen_kwargs)
    return _decode_clean(tokenizer, output_ids[0].tolist()).strip()
```

Figure A.1: Summarization function (src/summarizer.py)

Description: The summarize() function is the single entry point for every summarization request. It first checks the configured backend: when SUMMARY_PROVIDER is “gemini” it delegates to summarize_with_gemini() and returns the result. Otherwise, it loads the language-specific tokenizer and model, builds the generation settings (beam search, length limits, no-repeat n-grams), tokenizes and truncates the input, applies the IndicBART language-token format for Hindi, calls model.generate(), and decodes the output while stripping control tokens. The same function therefore serves both the FastAPI web application and the Streamlit demo.

A.2 OCR Routing

Source file: src/ocr.py

src/ocr.py

```python
def image_to_text_any(image=None, image_bytes=None,
                      mime_type="image/png", gpu=False):
    """Route OCR to Gemini vision (cloud) or EasyOCR (local)."""
    from .config import SUMMARY_PROVIDER
    if SUMMARY_PROVIDER == "gemini":
        from .llm_api import extract_text_with_gemini
        if image_bytes is None:
            import io
            buf = io.BytesIO()
            image.save(buf, format="PNG")
            image_bytes = buf.getvalue()
        return extract_text_with_gemini(image_bytes, mime_type)
    if image is None and image_bytes is not None:
        import io
        from PIL import Image
        image = Image.open(io.BytesIO(image_bytes))
    return image_to_text(image, gpu=gpu)
```

Figure A.2: OCR routing (src/ocr.py)

Description: image_to_text_any() abstracts over two OCR backends. In Gemini mode it forwards the raw image bytes to extract_text_with_gemini(), so no local OCR model is needed in the cloud deployment. In local mode it converts the bytes into a PIL image and calls image_to_text(), which runs EasyOCR for English and Hindi. This keeps the same UI code working in both the lightweight cloud build and the full local installation.

A.3 Image Summarization Endpoint

Source file: web/server.py

web/server.py

```python
@app.post("/api/ocr-summarize", response_model=SummaryResponse)
async def ocr_summarize(
    file: UploadFile = File(...),
    language: str = Form("en"),
    max_length: Optional[int] = Form(None),
    min_length: Optional[int] = Form(None),
) -> SummaryResponse:
    language = _validate_language(language)
    data = await file.read()
    try:
        ocr_text = image_to_text_any(image_bytes=data,
                                     mime_type=file.content_type or "image/png")
    except Exception as exc:
        raise HTTPException(status_code=400, detail=f"OCR failed: {exc}")
    if not ocr_text.strip():
        raise HTTPException(status_code=422, detail="No text detected")
    summary = summarize(ocr_text, language, max_length, min_length)
    return SummaryResponse(language=language, summary=summary,
                           source_text=ocr_text,
                           source_length=len(ocr_text))
```

Figure A.3: Image summarization endpoint (web/server.py)

Description: The /api/ocr-summarize endpoint accepts a multipart image upload together with the target language and length limits. It validates the language, reads the image bytes, extracts the text with image_to_text_any(), rejects the request when no text is detected, and then summarizes the extracted text. The response returns both the summary and the OCR text so the frontend can display them.

A.4 Gemini Vision Client

Source file: src/llm_api.py

src/llm_api.py

```python
def extract_text_with_gemini(image_bytes: bytes,
                             mime_type: str = "image/png") -> str:
    """OCR an image using Gemini's vision capability."""
    prompt = ("Extract all the text visible in this image. "
              "Return only the extracted text, preserving line breaks.")
    encoded = base64.b64encode(image_bytes).decode("ascii")
    return _generate(
        [
            {"text": prompt},
            {"inline_data": {"mime_type": mime_type, "data": encoded}},
        ],
        max_tokens=2048,
    )
```

Figure A.4: Gemini vision OCR client (src/llm_api.py)

Description: extract_text_with_gemini() implements OCR through Gemini's multimodal API. It builds an extraction prompt, base64-encodes the image, and sends it as inline image data with the correct MIME type. The shared _generate() helper posts the request to the Gemini REST endpoint and returns the recognized text, which is then passed to the summarization step.

APPENDIX B: ADDITIONAL SCREENSHOTS / OUTPUTS

Sample outputs generated by the system are shown below.

Table B.1: Sample Outputs

Language	Input (excerpt)	Generated summary (excerpt)
English	Artificial intelligence is transforming many industries…	Machine learning models can now read documents and images.
Hindi	भारत सरकार ने डिजिटल शिक्षा को बढ़ावा देने के लिए योजना…	भारत सरकार ने नई योजना की घोषणा की है।

OCR sample:

Input image (synthetic): “Hello from the OCR test document.”

OCR output: “Hello from the OCR test document.”

Generated summary: “This is an OCR test document.”